Day 6: Modernization Strategies & Migration Planning

---

1. Migration Phases

Definition

Migration is the process of moving applications from on-premises or legacy environments to cloud-based, modern platforms.

Key Phases

1. Assess

- Analyze current application architecture
- Identify dependencies, risks, and constraints

Example:

- Identify database, APIs, third-party services

---

2. Containerize

- Package application into containers
- Ensure portability across environments

---

3. Deploy

- Deploy containers to cloud platforms (GKE, Cloud Run)

---

4. Modernize

- Optimize architecture (microservices, serverless)
- Improve scalability, reliability, and performance

Scenario

Legacy HR System Migration

Problem:

- Runs on outdated hardware

- Difficult to scale

Solution:

- Assess → Containerize → Deploy → Modernize

Result:

- Improved performance and scalability

Case Study

Retail Organization

- Migrated inventory system to cloud

Result:

- Reduced infrastructure cost
- Faster deployments

---

2. Migration Strategies

---

1. Lift and Shift (Rehost)

Definition

Move application to cloud without major changes

Example:

- VM → Cloud VM

---

2. Replatform

Definition

Make minimal changes to optimize for cloud

Example:

- Move app to containers without redesign

---

3. Refactor (Re-architect)

Definition

Redesign application for cloud-native architecture

Example:

- Monolith → Microservices

Comparison

Strategy	Effort	Risk	Benefit
Lift & Shift	Low	Low	Quick migration
Replatform	Medium	Medium	Better performance
Refactor	High	High	Maximum scalability

Scenario

Banking Application Migration

- Quick migration needed → Lift & Shift
- Gradual improvement → Replatform
- Long-term transformation → Refactor

Case Study

Financial Services Company

- Phase 1: Lift & Shift
- Phase 2: Containerization
- Phase 3: Microservices

Result:

- Smooth transition with minimal risk

---

3. Application Dependency Mapping & Tooling

Definition

Identifying all components and dependencies of an application before migration.

Types of Dependencies

- Application services
- Databases
- External APIs
- Network dependencies

Tools

- Cloud-native discovery tools
- Monitoring tools
- Application profiling tools

Example

E-commerce application

Dependencies:

- Payment gateway
- Inventory database
- Notification service

Scenario

Unknown dependencies

Problem:

- Application fails after migration

Solution:

- Perform dependency mapping

Result:

- Smooth migration

Case Study

Enterprise ERP System

- Used dependency mapping tools

Result:

- Identified hidden dependencies
- Avoided downtime

---

4. Hybrid & Multi-Cloud Patterns

Hybrid Cloud

Definition

Combination of on-premises and cloud environments

Example:

- Database on-prem
- Application in cloud

Multi-Cloud

Definition

Using multiple cloud providers

Example:

- GCP + AWS

Benefits

- Avoid vendor lock-in
- Improve availability
- Disaster recovery

Scenario

Sensitive data restrictions

Problem:

- Data must stay on-prem

Solution:

- Use hybrid architecture

Result:

- Compliance maintained

Case Study

Healthcare Organization

- Uses hybrid cloud

Result:

- Secure data handling
- Scalable applications

---

5. Hands-On: Create Migration Plan

Step 1: Identify Application

Example:

- Legacy web application (3-tier architecture)

Step 2: Analyze Current State

- Application server
- Database server
- Dependencies

Step 3: Choose Strategy

- Lift & Shift / Replatform / Refactor

Step 4: Define Target Architecture

- Containers

- Kubernetes (GKE)
- Managed database

Step 5: Plan Migration Phases

1. Assessment
2. Containerization
3. Deployment
4. Optimization

Step 6: Risk Analysis

- Downtime
- Data loss
- Compatibility issues

Step 7: Create Timeline

- Week 1: Assessment
- Week 2: Containerization
- Week 3: Deployment
- Week 4: Testing

End-to-End Case Study (Day 6 Summary)

Scenario: Legacy E-Commerce Migration

Problem:

- Monolithic application
- High infrastructure cost
- Limited scalability

Solution

1. Assess current architecture

2. Choose replatform strategy
3. Containerize application
4. Deploy to Kubernetes
5. Gradually refactor to microservices

Result

- Scalable architecture
- Reduced cost
- Faster deployments
- Improved reliability